SIMATS ENGINEERING

ASSESSMENT TOOL 2

Case study

COURSE NAME: Computer Networks **COURSE CODE: CSA07**

COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Case study

QUESTIONS:4 **Total Marks:40**

1.A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an 8% packet loss, 180 ms jitter, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

Questions

1. Analyze whether the problem is primarily caused by the transport protocol (TCP/UDP) or the application layer.

2. Justify your answer based on the communication requirements of real-time multimedia applications.

3. Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.

2.A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the average DNS lookup

time is approximately 800 ms, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

Questions

1. Analyze the reasons behind the high DNS resolution time.

2. Propose a suitable DNS architecture using techniques such as Anycast DNS, DNS pre-fetching, and TTL optimization to reduce the lookup time below 50 ms.

3. Evaluate the advantages and limitations of your proposed architecture in terms of performance, scalability, and availability.

3.A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud hosted services. Investigation reveals that DNS queries are taking nearly 800 ms because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

Questions

4. Examine the impact of centralized DNS architecture on application performance.

5. Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.

6. Analyze how TTL values, Anycast routing, and DNS caching influence system performance and fault tolerance.

4.A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the average order acknowledgment latency increases from 1 ms to 40 ms, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

Questions

1. Analyze three possible TCP-related factors responsible for the latency increase, considering flow control, Nagle's algorithm, and SYN queue limitations.

2. Explain how each factor affects transaction processing in high-frequency systems

3. Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.

RUBRICS FOR CALCULATION

Criteria	**Weigh tag e**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developi ng**	**Level 1 – Beginning**
Understandi ng of Case	25	Thorough understandi ng; clearly identifies all key issues	Good understanding with most issues identified	Basic understan di ng; some issues missed	Poor understanding; problem not identified correctly
Application of Concept s	25	Effectively applies rele vant concepts to analyze the case deeply	Applies appropriate c oncepts with minor gaps	Limited applicatio n of concepts	Fails to apply relevant concepts

Analysis	20	In-depth analysis wit h strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited d epth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible sol utions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justificati on	Incorrect or no solution provided
Clarity	10	Clear, well structured, and easy to understand	Organized and understandabl e presentation	Limited clarity and struc ture	Poor clarity; difficult to understand

Case Study 1 — Video Conferencing Quality Issues

Q1. Transport protocol vs. application layer — Analysis

The symptoms — voice distortion, frozen frames, delayed communication, 8% packet loss and 180 ms jitter — point to a fault that spans both layers, but the transport-layer behaviour is the dominant cause:

- **Transport layer:** If the conferencing platform (or a fallback path such as a restrictive firewall/proxy) forces media over TCP instead of UDP, TCP's reliability mechanisms — retransmission, in-order delivery, and congestion-window back-off — convert every lost packet into an unpredictable delay. This is consistent with frozen frames and voice distortion during periods of loss.
- **Application layer:** A fixed or poorly-tuned jitter buffer cannot absorb 180 ms of jitter, and if it discards or delays audio/video frames arriving outside its window, that also produces distortion and freezing even when UDP is used correctly.

Diagnosis: The primary root cause is the transport protocol choice/behaviour (TCP-style reliability being applied to a loss-tolerant, delay-intolerant stream), with the application-layer buffering strategy acting as a secondary, compounding factor because it is not adaptive enough to compensate for the jitter that already exists on the network.

Q2. Justification based on real-time multimedia requirements

Real-time audio/video traffic has fundamentally different QoS requirements from typical data transfer:

- **Delay-sensitive, loss-tolerant:** Human perception tolerates a small amount of lost audio/video (concealed by codecs) far better than it tolerates delay. ITU-T recommends end-to-end one-way delay below ~150 ms for good conversational quality.
- **TCP is the wrong fit:** TCP guarantees reliability and ordering at the cost of retransmission delay and head-of-line blocking — a retransmitted packet arrives too late to be useful for a live conversation, so it only adds latency instead of improving quality.

- **UDP matches the requirement:** UDP has no retransmission or strict ordering, so late/lost packets are simply skipped or concealed, keeping the stream close to real time. This is why RTP (Real-time Transport Protocol) runs over UDP for VoIP/video conferencing.
- **Jitter's role:** 180 ms of jitter, if not smoothed by an adaptive buffer, causes frames to arrive out of cadence, which is perceived exactly as "frozen video" and "voice distortion."

Hence, both the observed metrics (loss, jitter) and the nature of the complaints are best explained by transport-layer mismatch, confirming the analysis in Q1.

Q3. Recommended improvements

Protocol-level improvements

- **Enforce UDP/RTP media transport:** Ensure the platform uses RTP over UDP for audio/video, reserving TCP only for signalling (SIP/HTTPS control messages).
- **RTCP feedback:** Use RTCP receiver reports to monitor loss/jitter in real time and drive adaptive behaviour.
- **Forward Error Correction (FEC):** Add redundant coding so the receiver can reconstruct moderately lost packets without retransmission.
- **Congestion control tuned for real time:** Use a delay-based scheme such as Google Congestion Control (GCC) or NADA instead of TCP-style loss-based back-off.
- **QoS marking / DiffServ (EF class) and traffic prioritization:** Prioritize RTP traffic over the WAN/VPN links used for conferencing.

Application-level improvements

- **Adaptive jitter buffering:** Dynamically resize the buffer based on measured jitter (e.g., 180 ms) instead of a fixed window.
- **Adaptive bitrate / codec switching:** Use codecs such as Opus (audio) and VP9/AV1 (video) that degrade gracefully and adjust bitrate to available bandwidth.

- **Packet loss concealment (PLC):** Interpolate missing audio samples and use frame extrapolation for video to mask short losses.
- **Bandwidth estimation and simulcast:** Send multiple quality layers so receivers with constrained links automatically fall back.

Case Study 2 — High DNS Resolution Time

Q1. Reasons behind the high DNS resolution time (~800 ms)

- **Centralized/distant authoritative servers:** Requests from geographically dispersed employees must travel to a single (or few) DNS servers, adding significant round-trip latency.
- **Deep recursive resolution chain:** Every lookup may traverse root → TLD → authoritative servers if results are not cached, multiplying RTTs.
- **Poor or absent caching / low TTLs:** Low TTL values force frequent re-resolution instead of reusing cached records.
- **No Anycast/CDN-based DNS:** Without Anycast, all users hit the same physical server regardless of location, ignoring proximity.
- **Single point of resolution / overloaded resolver:** A single DNS server handling global load can become a queuing bottleneck during peak usage.
- **No local caching/pre-fetching at the client or resolver level:** Every navigation triggers a fresh lookup instead of reusing recently resolved names.

Q2. Proposed DNS architecture (target: under 50 ms)

- **Anycast DNS:** Deploy the same DNS server IP address at multiple geographically distributed Points of Presence (PoPs). BGP routing automatically directs each user's query to the nearest instance, cutting RTT dramatically.
- **Tiered caching (recursive resolvers + local caching):** Deploy regional recursive resolvers close to users and enable aggressive, correctly-tuned local/browser DNS caching.

- **TTL optimization:** Set TTLs based on record volatility — e.g., 3600 s (1 hour) or higher for stable A/AAAA records instead of very low values — to maximize cache hits while still allowing timely updates during failover.
- **DNS pre-fetching:** Use browser/application-level DNS pre-fetch (e.g., <link rel="dns-prefetch">) and predictive pre-resolution of domains the user is likely to visit next, hiding lookup latency behind user think-time.
- **Redundant authoritative name servers:** Deploy multiple authoritative servers across providers/regions (multi-provider DNS) so a single server failure does not stop resolution — combined with health-checked failover.

Combined effect: Anycast removes the geographic distance penalty, caching/TTL tuning removes repeated full-resolution overhead, and pre-fetching hides remaining latency — together bringing effective lookup time from ~800 ms to well under 50 ms for the vast majority of queries.

Q3. Advantages and limitations of the proposed architecture

Performance

- Advantage: Anycast + caching drastically cuts round-trip time and server load; pre-fetching hides latency from the end user.
- Limitation: Aggressive caching/high TTLs can serve stale records for a period after legitimate IP changes.

Scalability

- Advantage: Anycast scales horizontally — adding PoPs increases capacity and improves proximity for new regions.
- Limitation: Requires BGP route management and coordination across multiple sites/providers, increasing operational complexity.

Availability

- Advantage: Multiple Anycast nodes and redundant authoritative servers mean a single node/server failure is masked automatically (traffic reroutes to the next-nearest healthy node).

- Limitation: Anycast failover during route convergence can briefly affect in-flight TCP-based DNS or session state; misconfigured BGP can cause traffic to be misdirected.

Case Study 3 — Distributed DNS for a Global Cloud Provider

Q1. Impact of centralized DNS architecture on application performance

- **Single point of latency:** All global queries converge on one primary server, so users far from that server always incur high propagation delay (~800 ms observed), regardless of local network quality.
- **Single point of failure:** If the primary server is overloaded or goes down, all dependent applications become unreachable — there is no automatic failover.
- **Poor load handling during traffic spikes:** During promotional events, sudden surges in simultaneous queries exceed the single server's processing/queueing capacity, causing intermittent delays and timeouts.
- **No traffic-aware routing:** A centralized design cannot route users to the nearest or least-loaded resource, so both DNS resolution and subsequent application access suffer.

Q2. Design of a distributed DNS solution

- **Anycast-based authoritative DNS:** Replace the single primary server with multiple authoritative servers sharing one Anycast IP, deployed across continents; BGP steers each query to the nearest healthy node.
- **Geo-distributed secondary/replica servers:** Use master–slave (primary–secondary) zone replication so every region has a full, up-to-date copy of the DNS zone.
- **Load-balanced recursive resolver tier:** Place regional recursive resolvers behind load balancers so query load is spread across many servers rather than one.

- **Health checks and automatic failover:** Continuously monitor node health; unhealthy nodes are withdrawn from Anycast advertisement so traffic is transparently redirected.
- **Multi-provider redundancy:** Use two independent DNS providers (e.g., diverse Anycast networks) so a provider-wide outage does not take down resolution entirely.

Q3. Influence of TTL, Anycast routing, and caching on performance and fault tolerance

TTL values

- Higher TTLs increase cache-hit rates and reduce repeated queries to authoritative servers, lowering average latency and server load.
- Lower TTLs improve fault tolerance and update agility (faster propagation of failover IP changes) but increase query volume and average latency — this is the classic performance-vs-agility trade-off, so TTLs should be tuned per record type (e.g., short TTL for failover records, long TTL for stable records).

Anycast routing

- Improves performance by directing users to the topologically nearest node, minimizing propagation delay.
- Improves fault tolerance because BGP naturally withdraws routes to failed nodes, redirecting traffic to the next-nearest healthy node without manual intervention.

DNS caching

- Reduces load on authoritative servers and shortens perceived lookup time for repeat queries, which is critical during traffic spikes such as promotional events.
- Over-caching can mask a failure (clients keep using a cached but now-invalid record) until the TTL expires, so caching must be balanced against TTL settings for records tied to failover.

Together, well-tuned TTLs, Anycast routing, and layered caching create a DNS system that is both fast under normal load and resilient during node failures or demand surges.

Case Study 4 — TCP Latency in a Stock Trading Platform

Q1 & Q2. TCP-related factors and their effect on high-frequency transaction processing

1. Flow control (receive window exhaustion)

Under peak load, if the receiving application (order-matching engine) cannot drain its socket buffer as fast as data arrives, its TCP receive window shrinks — potentially to zero (a "zero window" condition). The sender must then pause and periodically probe with window-probe segments before resuming transmission. In a high-frequency trading (HFT) system, this directly stalls order acknowledgments, turning a normally sub-millisecond round trip into tens of milliseconds while the sender waits for the window to reopen.

2. Nagle's algorithm

Nagle's algorithm delays sending small segments until either a full MSS worth of data accumulates or a pending ACK is received, in order to reduce the number of small packets on the network. Trading messages (order requests/acknowledgments) are typically small. When Nagle's algorithm is enabled on the sender and interacts with delayed ACKs on the receiver (which itself waits ~40 ms before ACKing a small segment to try to piggyback the ACK on outgoing data), the two mechanisms combine to produce a classic 'Nagle–delayed-ACK' deadlock-like delay — closely matching the observed jump to ~40 ms acknowledgment latency.

3. SYN queue limitations

Each new TCP connection begins with a SYN placed into the listening socket's SYN backlog queue while the three-way handshake completes. During market-open bursts, thousands of near-simultaneous connection attempts can overflow a small SYN backlog. Excess SYNs are silently dropped, forcing the client to retransmit the SYN after a timeout (often 1 second or more), which dramatically lengthens connection establishment time and delays the first order sent on that connection.

Combined effect on transaction processing

In an HFT environment where competitive advantage is measured in microseconds/milliseconds, all three factors compound: connections take longer to

establish (SYN queue), acknowledgments of individual orders are artificially delayed (Nagle/delayed-ACK), and bursts of orders can stall entirely when the receive window closes (flow control). The net effect is exactly what was observed — average acknowledgment latency rising from ~1 ms to ~40 ms, increased retransmissions (from SYN and data timeouts), and excessive buffering as the system tries to absorb the mismatch between offered load and effective throughput.

Q3. Recommended TCP configuration changes

- **Disable Nagle's algorithm:** Set the TCP_NODELAY socket option on trading connections so small order messages are sent immediately instead of being batched.
- **Disable or reduce delayed ACK:** Tune TCP_QUICKACK (Linux) or reduce the delayed-ACK timer so acknowledgments are not held back, removing the Nagle/delayed-ACK interaction.
- **Increase the SYN backlog:** Raise net.core.somaxconn and net.ipv4.tcp_max_syn_backlog, and enable SYN cookies (tcp_syncookies) so legitimate bursts of connection attempts are not dropped during market open.
- **Tune socket buffer sizes and window scaling:** Increase send/receive buffer sizes (net.ipv4.tcp_rmem / tcp_wmem) and ensure TCP window scaling is enabled so the flow-control window is large enough to sustain high-throughput bursts without stalling.
- **Use persistent/pooled connections:** Avoid repeated connection setup/teardown for each order by keeping long-lived, pre-established TCP connections (or connection pools) between trading clients and the matching engine.
- **Consider low-latency transport alternatives:** For the most latency-critical paths, evaluate kernel-bypass networking (DPDK, RDMA/RoCE) or multicast-based market data feeds, which avoid standard TCP stack overhead altogether.
- **Enable TCP_NODELAY with careful batching logic at the application layer:** Where some batching is still desirable for efficiency, implement it explicitly in the application (not via Nagle) so batching decisions are latency-aware rather than fixed by the OS default.